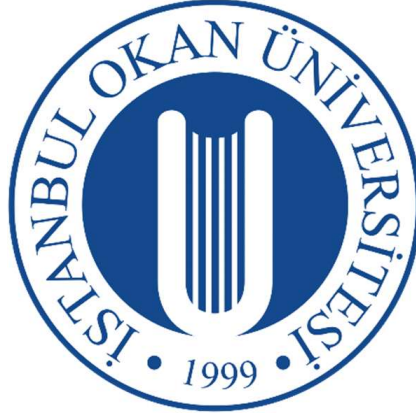


**FACULTY OF APPLIED SCIENCES
MANAGEMENT INFORMATION SYSTEMS**



**CORPORATE COMMUNICATION AND EMPLOYEE
MANAGEMENT
PLATFORM: CORPCONNECT**

GRADUATION PROJECT

[YOUR FULL NAME]

[MONTH YEAR]



FACULTY OF APPLIED SCIENCES
MANAGEMENT INFORMATION SYSTEMS

CORPORATE COMMUNICATION AND EMPLOYEE MANAGEMENT
PLATFORM: CORPCONNECT

GRADUATION PROJECT

[YOUR FULL NAME]

Submitted in partial fulfillment of the requirements for the degree of Bachelor of Science.

Advisor:
[ADVISOR TITLE AND NAME]

[MONTH YEAR]

TABLE OF CONTENTS

LIST OF FIGURES	i
LIST OF TABLES	ii
LIST OF ABBREVIATIONS	iii
ABSTRACT	iv
ACKNOWLEDGEMENTS	v
1. Introduction	1
2. Literature Review	2
2.1. Existing Industry Solutions	2
2.2. Academic Studies	2
2.3. Distinctive Aspects of CorpConnect	2
3. System Design and Technology Stack	4
3.1. System Architecture	4
3.2. Server-Side Technologies	5
3.3. Client-Side Technologies	5
3.4. Data Model	6
3.4.1. Key Relationships Between Entities	7
3.5. Security and Authorization	7
3.5.1. Authentication	7
3.5.2. Authorization	8
3.5.3. Data Security and Communication	8
4. Application Development and Features	9
4.1. User Management and Roles	9
4.1.1. Login, Logout, and Password Reset	9
4.1.2. Profile Management	9
4.2. Messaging Module	9
4.3. Task Management	10
4.3.1. Leave Management	10
4.3.2. Expense Management	10
4.4. Knowledge and Social Modules	11
4.4.1. News and Announcements	11
4.4.2. Polls	11
4.4.3. Training	11
4.4.4. Calendar	11
4.4.5. Suggestion Box	11
4.4.6. Celebrations	11

4.5. Administration Panel	11
5. Conclusion and Recommendations	13
5.1. Limitations	13
5.2. Recommendations for Future Work	14
REFERENCES	15
APPENDICES	17

LIST OF FIGURES

Figure 3.1	CorpConnect Three-Tier System Architecture	4
Figure 3.2	Data Model Entity-Relationship (ER) Diagram	6
Figure 4.3	Messaging Module Screenshot	9
Figure 4.2	Task Kanban Board Screenshot	10

LIST OF TABLES

Table 3.1	Server-Side Technology Stack	5
Table 3.2	Client-Side Technology Stack	5
Table 3.3	Database Tables Overview	6
Table 4.1	Role-Based Permission Matrix	8
Table 4.2	Module–Feature Mapping	11

LIST OF ABBREVIATIONS

API	Application Programming Interface
CORS	Cross-Origin Resource Sharing
CRUD	Create, Read, Update, Delete
CSP	Content Security Policy
ER	Entity-Relationship
HMR	Hot Module Replacement
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
JWT	JSON Web Token
MIS	Management Information Systems
MVC	Model-View-Controller
ORM	Object-Relational Mapping
PDPL	Personal Data Protection Law
RBAC	Role-Based Access Control
REST	Representational State Transfer
SMB	Small and Medium-Sized Business
SPA	Single Page Application
SQL	Structured Query Language
UI/UX	User Interface / User Experience
XSS	Cross-Site Scripting

ABSTRACT

CORPORATE COMMUNICATION AND EMPLOYEE MANAGEMENT PLATFORM: CORPCONNECT

Today, companies must carry out inter-employee communication, workflow tracking, and organizational knowledge sharing in digital environments. Existing solutions in the market tend to focus on a single need — such as messaging only, task management only, or human resources operations only — forcing organizations to use multiple tools simultaneously. This situation creates disadvantages in terms of both cost and efficiency. **CorpConnect**, developed within the scope of this study, brings together modules including instant messaging, task assignment and tracking, leave request and approval workflows, expense management, corporate news, polls, training materials, calendar, a suggestion system, and special-day celebrations into a single web-based platform. Node.js, Express, Prisma ORM, and MySQL were used on the server side, while React 19, Vite, and Tailwind CSS were used on the client side. Real-time communication is provided through Socket.IO, and authentication is implemented using a JSON Web Token-based short-lived access and long-lived refresh token pair. Three roles are defined in the system (Administrator, Department Manager, Employee), with role-based authorization enforced throughout. As a result, a modular and extensible corporate intranet platform has been produced, enabling small and medium-sized businesses to manage communication and processes from a single point.

Keywords: Corporate Intranet, Full-Stack Web Development, Real-Time Communication, Task Management, Role-Based Access Control.

ACKNOWLEDGEMENTS

I would like to express my sincere gratitude to my advisor, [ADVISOR TITLE AND NAME], for their invaluable guidance, constructive feedback, and continuous support throughout the development of this project. I am grateful to all the faculty members of the department who contributed to my academic and professional development during my undergraduate education. I also thank my friends who provided feedback during the preparation and testing phases of this work, and my family, who stood by me at every step of the way.

1. INTRODUCTION

The impact of information and communication technologies on the business world is growing with each passing day. As hybrid and remote working models have become widespread—particularly in the post-pandemic era—conducting inter-employee communication and managing workflows through digital platforms has become a necessity. In this process, companies are compelled to use multiple tools simultaneously, such as e-mail, instant messaging applications, task tracking systems, and leave management software. This situation both increases licensing costs and makes it more difficult to maintain information integrity.

The aim of this study is to develop a modular and scalable application, built on modern web technologies, that brings together the various modules needed by small and medium-sized businesses (SMBs) into a single corporate intranet platform. The developed platform aims to provide different business needs—including instant messaging, task management, leave and expense requests, corporate news, polls, training, calendar, and social modules (suggestion box, birthday and anniversary celebrations)—within a unified hub.

The main motivations for choosing this topic can be summarized as follows: (i) the majority of integrated solutions on the market are of foreign origin and Turkish language support tends to be secondary; (ii) licensing costs for such solutions are high for SMB-scale organizations; (iii) the efficiency loss caused by information fragmentation across different tools. The developed platform, offered in English with an open-source architecture, aims to address these shortcomings.

Contributions of this project to the field of information technology include: providing a practical example of how a full-stack web application can be designed in a three-tier architecture; demonstrating how a WebSocket-based stream for real-time communication is integrated; and offering a concrete reference for how role-based access control (RBAC) can be applied on a modular intranet.

The remaining sections of this work are organized as follows: **Chapter 2** reviews similar products used in the industry and academic studies, identifying the ways in which CorpConnect differs from existing solutions. **Chapter 3** describes the system’s design decisions, the technology stack employed, the data model, and the security mechanisms. **Chapter 4** details the platform’s modules, screen flows, and the development process. **Chapter 5** evaluates the results obtained and presents the limitations of the project along with recommendations for future development.

2. LITERATURE REVIEW

Corporate communication and employee management platforms have attracted the attention of both academic communities and industry players for approximately fifteen years. This chapter first examines products commonly used in the industry, then summarizes findings from academic sources, and finally explains CorpConnect's position within this landscape.

2.1. Existing Industry Solutions

Microsoft Teams and **Slack** are solutions centered on instant messaging and audio/video communication, built around a corporate channel structure. Although both offer rich integration ecosystems, task management and human resources workflows can only be added through third-party applications. **Workplace by Meta** (discontinued in 2024) brought a Facebook-like social feed model to the enterprise, but remained limited in business process management. **Asana** and **Trello** specialize exclusively in task and project management and do not cover areas such as communication, leave, or expenses. In the Turkish market, intranet solutions such as **Mobilyaka** address basic needs like news, organizational charts, and announcements, but offer limited room for modular expansion.

2.2. Academic Studies

In the relevant academic literature, the concept of Enterprise Social Software is addressed in the context of its effects on employee engagement, knowledge sharing, and intra-organizational communication. Leonardi et al. (2013) demonstrated that enterprise social platforms increase knowledge visibility and thereby improve collaboration among employees. Pirkkalainen and Pawlowski (2014) examined success factors related to user acceptance of multi-module enterprise software, identifying ease of use, mobile accessibility, and process integration as primary elements. More recent studies discuss the performance advantages of **WebSocket** and **Socket.IO**-based architectures for real-time communication infrastructure compared to HTTP polling approaches.

2.3. Distinctive Aspects of CorpConnect

Upon reviewing the above products and studies, the key features that distinguish CorpConnect can be summarized as follows:

- **Integrated structure:** Consolidates diverse needs—communication, tasks, leave, expenses, calendar, suggestions, training, and celebrations—into a single platform.
- **Modern open-source stack:** Built on widely adopted, free, and sustainable technologies such as Node.js, React 19, Prisma ORM, and MySQL.
- **Role-based modular authorization:** Fine-grained permission separation is imple-

mented among three roles: Administrator, Department Manager, and Employee.

- **Activity log (audit trail):** All write operations are centrally logged to produce a complete audit trail.

In these respects, CorpConnect is positioned as a modular and extensible reference application that offers an alternative to the single-focus solutions prevalent in the industry and is suited to the scale of SMBs.

3. SYSTEM DESIGN AND TECHNOLOGY STACK

This chapter covers CorpConnect’s design decisions, its three-tier architectural approach, the technologies used on the server and client sides, the data model, and the security mechanisms.

3.1. System Architecture

CorpConnect is built on the classic three-tier architecture:

1. **Presentation Layer (Client):** A Single Page Application (SPA) developed with React 19. It runs in the browser and presents an English-language interface to the user.
2. **Application Layer (Server):** A REST API written with the Express framework on Node.js. It executes business rules, performs authentication, and broadcasts real-time notifications via Socket.IO.
3. **Data Layer:** MySQL 8 relational database. Queried by the application in a schema-safe manner through Prisma ORM.

Inter-layer communication is carried out over REST API endpoints (HTTP) and Web-Socket (Socket.IO) for real-time events. The client and server run on different ports; in the development environment, CORS is transparently handled through Vite’s proxy configuration, while in production the client build can be served as static files.

[Figure 3.1 to be inserted] Three-tier architecture diagram: Browser (React) → Express API → MySQL boxes with a Socket.IO bidirectional arrow.

Figure 3.1 CorpConnect Three-Tier System Architecture

A layered file structure similar to the classic **MVC** pattern is followed on the server side:

- routes/ — HTTP endpoint definitions
- validations/ — Request validation chains with express-validator
- controllers/ — Business logic and Prisma calls
- middleware/ — Authentication, authorization, validation, file upload, global error handling

- `utils/` — Pagination, JWT, bcrypt, response envelope, activity log helpers
- `config/` — CORS, Helmet, multer, Socket.IO configurations
- `socket/handlers/` — Real-time event handlers

On the client side, pages are organized under `pages/`, shared components under `components/`, API calls under `services/`, global state management under `context/`, and validation schemas under `schemas/`.

3.2. Server-Side Technologies

The technologies selected for the server layer and the rationale for their selection are summarized in Table 3.1.

Table 3.1 Server-Side Technology Stack

Technology	Version	Role
Node.js	24.12	JavaScript runtime environment
Express	4.18	HTTP routing and middleware framework
Prisma ORM	5.22	Database access layer, schema migrations
MySQL	8.0	Relational database management system
Socket.IO	4.6	Real-time communication (WebSocket)
jsonwebtoken	9.0	JSON Web Token generation and verification
bcryptjs	2.4	Password hashing
Helmet	7.0	Security headers (CSP, HSTS)
multer	1.4	Multipart file upload handling
Winston	3.19	Configurable logging
express-rate-limit	7.0	Request rate limiting

Node.js, with its event-loop-based, asynchronous, single-threaded execution model, can efficiently handle a large number of concurrent connections. This characteristic proved decisive in scenarios requiring persistently open connections, such as messaging and notifications. Express enables rapid endpoint development thanks to its minimalist structure and large ecosystem. Prisma increases developer productivity by providing type-safe query methods rather than handwritten SQL.

3.3. Client-Side Technologies

React 19 was chosen as the foundation on the client side, with Vite selected as the modern build tool. Tailwind CSS v4 was used for style management through its utility-first approach.

Table 3.2 Client-Side Technology Stack

Technology	Version	Role
React	19.x	Component-based UI library
Vite	5.x	Development server and module bundler
react-router	7.x	Client-side routing
Tailwind CSS	4.x	Utility-first styling framework
axios	—	HTTP client (with interceptor support)
react-hook-form	—	Form state management
Zod	—	Schema-based validation
date-fns	—	Date formatting (locale support)
lucide-react	—	Icon library
react-hot-toast	—	Notification toasts
socket.io-client	—	Real-time client

Vite’s Hot Module Replacement (HMR) feature accelerated the development cycle. Tailwind CSS’s utility-first approach minimized custom CSS writing while maintaining a consistent design language. React Router’s nested route support allowed the two separate layout trees for regular users and super administrators to be clearly separated.

3.4. Data Model

The CorpConnect data model is grouped into five main categories: user, communication, business process, social interaction, and management. The database schema is defined declaratively in Prisma’s `schema.prisma` file at a single point and placed under version control through migration files.

[Figure 3.2 to be inserted] Data model entity-relationship (ER) diagram. Key entities: User, UserProfile, Conversation, Message, Task, TaskComment, LeaveRequest, Expense, CalendarEvent, Suggestion, Notification, ActivityLog.

Figure 3.2 Data Model Entity-Relationship (ER) Diagram

Table 3.3 Database Tables Overview

Group	Tables	Description
User	users, user_profiles, refresh_tokens, password_resets	Identity, profile, session management
Communication	conversations, conversation_members, messages	One-on-one and group messaging
Notification	notifications, notification_preferences	Real-time alerts, user preferences
Task	tasks, task_comments	Task creation, assignment, commenting
Leave	leave_requests	Leave application and approval flow
Expense	expenses	Expense reporting and approval
Calendar	calendar_events, event_attendees	Event and attendee management
Knowledge	news, polls, poll_votes, trainings	News, polls, training materials
Social	suggestions, celebrations	Suggestion box, celebrations
Management	activity_logs, system_settings	Audit trail, system settings

3.4.1. Key Relationships Between Entities

The User entity is at the center of the system and connects to other entities through numerous relationships, including tasks it created and was assigned to (`createdTasks`, `assignedTasks`), messages it sent (`sentMessages`), leave requests (`leaveRequests`), expense records (`expenses`), and the activity log (`activityLogs`). The relationship between Conversation and User is established as many-to-many through the `ConversationMember` junction table, allowing both one-on-one and group conversations to be modeled within a single structure. All write operations are recorded in the `ActivityLog` table, creating a complete audit trail.

3.5. Security and Authorization

In the CorpConnect design, security was treated as a quality attribute considered from the outset, rather than a byproduct of the architecture.

3.5.1. Authentication

Authentication is implemented using a JSON Web Token (JWT) dual-token approach:

- **Access Token:** Short-lived (15 minutes), carried in the Authorization header with every HTTP request.
- **Refresh Token:** Long-lived (7 days), used to generate a new access token when the

current one expires, and stored in the database.

Passwords are hashed and stored using the bcrypt algorithm with 10 rounds. Plain-text passwords or reversible encryption were not used.

3.5.2. Authorization

Three roles exist in the system:

Table 4.1 Role-Based Permission Matrix

Capability	Admin	Dept. Manager	Employee
Access super admin panel	✓	×	×
User management	✓	×	×
Modify system settings	✓	×	×
Approve leave requests	✓	✓	×
Approve expenses	✓	✓	×
Create and assign tasks	✓	✓	✓
Messaging	×*	✓	✓
Create news, announcements, polls	✓	✓	×

*The Administrator role is designed for system monitoring purposes only and is hidden from user-facing lists.

Authorization is enforced end-to-end on the server side through the `auth.middleware` (authentication) and `role.middleware` (role checking) middleware layers.

3.5.3. Data Security and Communication

- **CORS:** Only the defined client origin is permitted.
- **Helmet:** Browser security headers (Content Security Policy, HTTP Strict Transport Security, etc.) are added automatically.
- **Rate Limiting:** A limit of 10 attempts per 15 minutes is applied to `/auth` endpoints.
- **File Upload:** A maximum of 10 MB and only permitted MIME types are accepted, managed by Multer.
- **Input Validation:** express-validator chains are executed on every write endpoint.

4. APPLICATION DEVELOPMENT AND FEATURES

This chapter covers the modules of the CorpConnect platform, usage flows, and key decisions made during development.

4.1. User Management and Roles

The user model includes e-mail, password, first name, last name, role, activity status, and profile extension fields (department, position, avatar, phone number, biography, date of birth, employment start date). Profile information is stored in a separate `UserProfile` table and linked via a one-to-one relationship, so that the user can be represented as a complete entity.

4.1.1. Login, Logout, and Password Reset

Users log in by submitting their e-mail and password to the `POST /auth/login` endpoint; on success, an access token and a refresh token are returned. The refresh token is persisted in the `refresh_tokens` table, enabling re-authentication without requiring login credentials again. Logout invalidates the current refresh token by deleting it from the database, so the token cannot be reused even if it is intercepted. For password reset, a time-limited token is sent to the registered e-mail address; only after that token is verified is the user permitted to create a new password.

4.1.2. Profile Management

Each user can update their personal information (name, position, department, avatar, biography, phone) and security credentials (password). Avatar uploads pass through the Multer middleware, the file is saved to the server, and the URL is stored in `UserProfile`.

4.2. Messaging Module

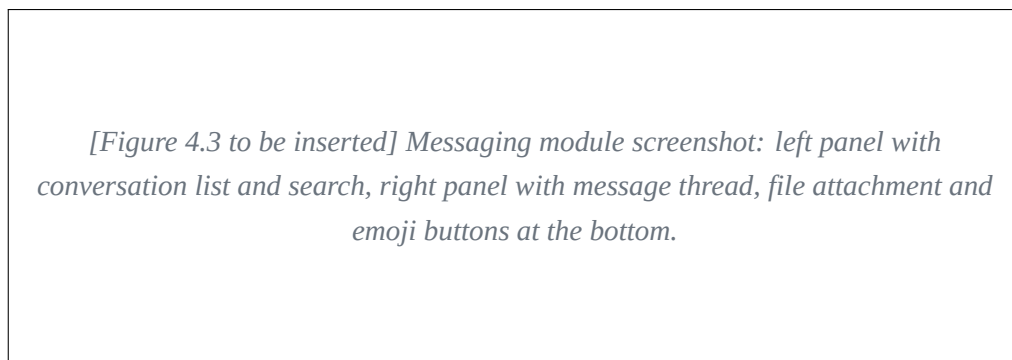


Figure 4.3 Messaging Module Screenshot

The `Conversation` entity supports both one-on-one and group conversations. The `ConversationMember`

table tracks which users belong to which conversation, and the Message entity stores the message content, sender, and file attachments. Messages are delivered in real time through Socket.IO. Users can also react to messages, tag specific messages, and archive conversations.

4.3. Task Management

The task module is built on the classic Kanban board (To Do → In Progress → Done → Cancelled). Each task contains a title, description, priority (Low, Normal, High, Urgent), status, due date, and assignee.

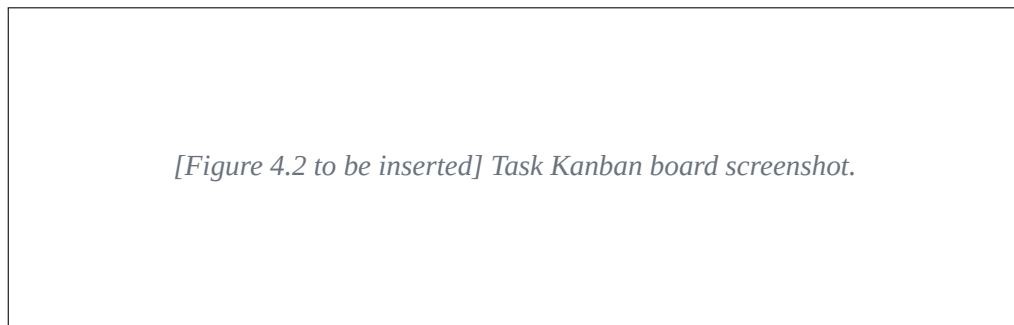


Figure 4.2 Task Kanban Board Screenshot

The task detail page is designed with a two-column layout containing a description, quick status-transition buttons, a comment thread (TaskComment), and an activity history (fed from ActivityLog). The assignment component is implemented as a searchable popover that displays avatar and department information, similar to the user-selection pattern in the messaging module. The Administrator (ADMIN) role is excluded from the assignable user list.

4.3.1. Leave Management

The LeaveRequest entity includes fields for leave type (annual, sick, excuse, unpaid), start and end dates, status (Pending, Approved, Rejected, Cancelled), reviewer, and review notes. The employee creates the request; the relevant department manager or system administrator approves or rejects it. A notification is sent to the user upon approval.

4.3.2. Expense Management

The Expense entity includes title, description, amount, currency, expense date, receipt attachment, status, and reviewer fields. Receipts are linked through the file upload module (File). The approval flow follows the same pattern as leave requests.

4.4. Knowledge and Social Modules

4.4.1. News and Announcements

The `News` entity includes title, content, category, cover image, publication status, and publication date fields. Administrators and department managers can create news items; employees can only read published ones.

4.4.2. Polls

The `Poll` and `PollOption` entities model multiple-choice polls. Each user can vote in a given poll only once; votes are stored in the `PollVote` table.

4.4.3. Training

The `Training` entity includes fields for training title, description, material link (video or document), and completion status. Employees can review training materials assigned to them.

4.4.4. Calendar

The `CalendarEvent` and `EventAttendee` entities contain fields for event title, type (meeting, birthday, training, holiday), start and end times, location, online meeting link, organizer, and attendee list. Attendee status (Attending, Maybe, Not Attending) is stored in a separate field.

4.4.5. Suggestion Box

The `Suggestion` entity includes title, content, category, status, anonymous submission option, and manager note fields. Employees can submit suggestions regarding organizational processes; suggestions are reviewed by managers.

4.4.6. Celebrations

Special occasions such as birthdays and employment anniversaries are highlighted on the home page through the `Celebration` module, strengthening employee engagement and social interaction.

4.5. Administration Panel

The `/superadmin` layout reserved for the Administrator role contains a **Dashboard** page showing system-wide statistics, user management (`Users`), leave approvals (`Leaves`), expense approvals (`Expenses`), activity log (`Logs`), reports (`Reports`), suggestion management (`Suggestions`), corporate broadcast (`Broadcast`), and system settings (`Settings`). The sidebar structure is kept separate from the regular user layout to reduce cognitive load.

Table 4.2 Module–Feature Mapping

Module	Key Features
Identity Management	JWT, password reset, session refresh
Messaging	1-on-1 + group, file attachment, reactions, tagging, archive
Notification	Type-based preferences, real-time delivery
Task	Kanban, assignment, comments, activity history, due-date alerts
Leave	Multiple types, approval flow, balance tracking
Expense	Receipt upload, approval workflow, VAT field
Calendar	Events, attendee status, category filter
News / Poll / Training	Publication status, categories, view tracking
Suggestion / Celebration	Anonymous suggestions, automatic celebration reminders
Admin Panel	Users, logs, reports, settings

5. CONCLUSION AND RECOMMENDATIONS

Within the scope of this study, CorpConnect — a modular and extensible corporate intranet application capable of meeting the communication, workflow tracking, and employee interaction needs of small and medium-sized businesses within a single platform — has been designed and developed. The key outcomes of this work can be summarized as follows:

- A sustainable, clearly responsibility-separated codebase has been produced using a three-tier architecture.
- A stateless, horizontally scalable server infrastructure has been designed using JSON Web Token-based authentication.
- Thanks to role-based authorization middleware, all modules can be fine-grained controlled from a single central point.
- Real-time messaging and notification delivery via Socket.IO has enabled instant interaction among employees.
- The activity log, in which all write operations are centrally recorded, fulfills the audit trail requirement.

In terms of contribution to the field of information technology, CorpConnect offers a concrete reference for how a full-stack intranet application can be designed within the Node.js and React 19 ecosystem. Its English-language interface and architecture that allows modular growth can be considered as an alternative to foreign solutions in the market for organizations at the SMB scale.

5.1. Limitations

The current version of the developed system has been produced within the scope of an undergraduate capstone project, and the following limitations should be taken into account:

- **Test Coverage:** Automated unit and integration tests have not yet been written; verification has been limited to manually conducted smoke tests.
- **Mobile Application:** Only a responsive web interface exists; native iOS/Android applications have not been developed.
- **Scalability:** Tested on a single server instance and single MySQL instance; performance measurements under high concurrent load have not been conducted.
- **Internationalization:** Only an English interface is provided; the infrastructure for multi-language support has not been built.
- **Artificial Intelligence:** AI-assisted capabilities such as smart recommendations, automatic summarization, or natural language querying have not been included in scope.

5.2. Recommendations for Future Work

This study provides a foundation amenable to development in the following directions:

1. **Automated Test Infrastructure:** Adding server-side integration tests with tools such as Jest and Supertest, and end-to-end tests with Playwright or Cypress.
2. **Mobile Client:** Developing a React Native-based mobile application that consumes the same API.
3. **AI-Assisted Modules:** Large language model (LLM) integration for capabilities such as task recommendations, e-mail or message summarization, and anonymous sentiment analysis.
4. **PDPL Compliance Module:** Systematizing personal data inventory, explicit consent management, and data deletion request workflows.
5. **Microservice Migration:** Separating high-traffic modules (messaging, notifications) into independent services and integrating them with a message queue (e.g., RabbitMQ).
6. **Multi-Tenant Architecture:** Converting to a SaaS model capable of serving multiple companies from a single installation.
7. **Advanced Analytics:** Dashboards presenting employee engagement, task completion rates, and interaction indicators to managers.

These recommendations outline the steps needed for CorpConnect to evolve from an academic prototype into an enterprise-grade product.

REFERENCES

- Microsoft. (2025). *Microsoft Teams documentation*. Retrieved from: <https://learn.microsoft.com/microsoftteams/>
- Slack Technologies. (2025). *Slack platform documentation*. Retrieved from: <https://api.slack.com/>
- Meta Platforms. (2024). *Workplace transition announcement*. Retrieved from: <https://www.workplace.com/>
- Atlassian. (2025). *Trello & Jira Software resources*. Retrieved from: <https://www.atlassian.com/>
- Asana. (2025). *Asana developer documentation*. Retrieved from: <https://developers.asana.com/>
- Node.js Foundation. (2025). *Node.js documentation*. Retrieved from: <https://nodejs.org/docs/>
- OpenJS Foundation. (2025). *Express.js documentation*. Retrieved from: <https://expressjs.com/>
- Meta. (2025). *React documentation (v19)*. Retrieved from: <https://react.dev/>
- Vite Project. (2025). *Vite documentation*. Retrieved from: <https://vitejs.dev/>
- Prisma. (2025). *Prisma ORM documentation*. Retrieved from: <https://www.prisma.io/docs/>
- Tailwind Labs. (2025). *Tailwind CSS documentation (v4)*. Retrieved from: <https://tailwindcss.com/>
- Socket.IO Team. (2025). *Socket.IO documentation*. Retrieved from: <https://socket.io/docs/>
- Leonardi, P. M., Huysman, M., & Steinfield, C. (2013). Enterprise social media: Definition, history, and prospects for the study of social technologies in organizations. *Journal of Computer-Mediated Communication*, 19(1), 1–19.
- Pirkkalainen, H., & Pawlowski, J. M. (2014). Global social knowledge management — understanding barriers for global workers utilizing global social software. *Computers in Human Behavior*, 30, 637–647.
- Riemer, K., & Tavakoli, A. (2013). The role of groups as local context in large enterprise social networks: A case study of Yammer at Deloitte Australia. *Business Information Systems Engineering*, 5(5), 327–340.
- Treem, J. W., & Leonardi, P. M. (2012). Social media use in organizations: Exploring the affordances of visibility, editability, persistence, and association. *Communication Yearbook*, 36, 143–189.

OWASP Foundation. (2021). *OWASP Top 10:2021*. Retrieved from: <https://owasp.org/Top10/>

APPENDICES

APPENDIX 1: Database Enum Values

Enum	Values
Role	ADMIN, MANAGER, EMPLOYEE
TaskPriority	LOW, NORMAL, HIGH, URGENT
TaskStatus	TODO, IN_PROGRESS, DONE, CANCELLED
LeaveType	ANNUAL, SICK, EXCUSE, UNPAID
LeaveStatus	PENDING, APPROVED, REJECTED, CANCELLED
ExpenseStatus	PENDING, APPROVED, REJECTED
NotificationType	NEWS, POLL, MESSAGE, TASK, LEAVE, EXPENSE, EVENT, SUGGESTION, CELEBRATION
SuggestionCategory	PROCESS, ENVIRONMENT, BENEFITS, TECHNOLOGY, OTHER
SuggestionStatus	PENDING, UNDER_REVIEW, APPROVED, REJECTED, IMPLEMENTED
EventType	MEETING, BIRTHDAY, TRAINING, HOLIDAY, OTHER

APPENDIX 2: Sample Code Snippets

Authentication Middleware (auth.middleware.js)

```
const authenticate = async (req, res, next) => {
  try {
    const authHeader = req.headers.authorization
    if (!authHeader || !authHeader.startsWith('Bearer ')) {
      return error(res, 'Token not found', 401)
    }
    const token = authHeader.split(' ')[1]
    const decoded = verifyAccessToken(token)
    const user = await prisma.user.findUnique({
      where: { id: decoded.userId },
      select: { id: true, email: true, firstName: true, lastName: true, role: true, isActive: true, deletedAt: true },
    })
    if (!user || !user.isActive || user.deletedAt) return error(res, 'Account not active', 401)
    req.user = user
    next()
  } catch (err) {
    if (err.name === 'TokenExpiredError') return error(res, 'Token expired',
```

```

    401)
    if (err.name === 'JsonWebTokenError') return error(res, 'Invalid token',
    401)
    return error(res, 'Authentication error', 500)
  }
}

```

Task Detail and Comment List (task.controller.js — getTaskById)

```

const getTaskById = async (req, res) => {
  const id = parseInt(req.params.id)
  const task = await prisma.task.findFirst({ where: { id, deletedAt: null },
    select: taskSelect })
  if (!task) return error(res, 'Task not found', 404)
  if (req.user.role === 'EMPLOYEE' && task.creator.id !== req.user.id && task.
    assignee?.id !== req.user.id) {
    return error(res, 'You do not have permission to view this task', 403)
  }
  const [comments, activity] = await Promise.all([
    prisma.taskComment.findMany({
      where: { taskId: id, deletedAt: null },
      orderBy: { createdAt: 'asc' },
      select: { id: true, body: true, createdAt: true, updatedAt: true, user: {
        select: userMini } },
    })),
    prisma.activityLog.findMany({
      where: { entity: 'task', entityId: id },
      orderBy: { createdAt: 'desc' },
      take: 50,
      select: { id: true, action: true, detail: true, createdAt: true, user: {
        select: { id: true, firstName: true, lastName: true } } },
    })),
  ])
  return success(res, { ...task, comments, activity }, 'Task retrieved')
}

```

Prisma Schema — Task Comment

```

model TaskComment {
  id          Int          @id @default(autoincrement())
  taskId      Int
  userId      Int
  body        String       @db.VarChar(2000)
  deletedAt   DateTime?
  createdAt   DateTime      @default(now())
  updatedAt   DateTime      @updatedAt

  task Task @relation(fields: [taskId], references: [id], onDelete: Cascade)
  user User @relation(fields: [userId], references: [id], onDelete: Cascade)

  @@index([taskId, createdAt])
}

```

```
    @@index([userId])  
    @@map("task_comments")  
}
```